Lab 25: /find-callers needs Grep with a glob, not Bash

Topic 2: Tool Design & MCP Integration · Task 2.5: Select and apply built-in tools (Read, Write, Edit, Bash, Grep, Glob) effectively

Runs on Simulator 4: Developer Productivity with Claude · Run in class

Source: SkillCertPro practice exam Part 1, Q37 (unofficial third-party question)

The question

A payments platform's developer-productivity team is adding a /find-callers command that should locate every reference to a renamed helper function, limit the search to TypeScript source files, and return enough context for a developer to jump directly to each match. Which Claude Code built-in tool configuration should the command use?

A. Use Glob with pattern **/helperName.ts , sort the returned paths, and treat matching file names as caller locations.

B. Use Glob with pattern **/*.ts , then Read each file and load all contents into the conversation for scanning.

C. Use Bash to run grep -r helperName --include=*.ts , then parse stdout as caller locations.

D. Use Grep with the helper name as pattern, scope it with glob **/*.ts , and set output_mode: "content" for match lines.

Simulate the solution

```
# In the browser: open Simulator 4, then Lab 25
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 25

# In a terminal, from simulators/simulator-4-developer-productivity/
uv run explorer.py --case UC-37
```

What you should see

One Grep call for calculate_total in *.py, with file, line and match. No Bash grep.

One Grep call with pattern calculate_total, glob *.py and output_mode content; no Glob-then-Read-everything and no Bash grep.

Open `simulators/simulator-4-developer-productivity/evidence/` and find it yourself.

The answer

D. Grep with the helper name, a file-type glob, output_mode content.

Grep is the built-in tool for searching file contents such as function names, error messages and imports. Callers of a helper are content inside files. Glob finds files by path or name pattern (for example **/*.test.ts). Grep with output_mode: "content" returns matching lines with file and line number, and its glob filter limits the search to TypeScript. The built-in Grep tool is not the same as shell grep through Bash, which goes through Bash permissions.

Why the others fail

A. Matches file names . A file called helperName.ts is the definition, not its callers.

B. Loads every file into context, which is wasteful and pointless when Grep exists.

C. Works, but bypasses the purpose-built tool, depends on Bash permissions, and leaves fragile output parsing.

Check yourself

- Grep used with pattern == helper name
- Grep scoped with a *.py glob and output_mode content
- no Bash grep and fewer than 3 Read calls

Now change one thing

Edit the prompt_template in this lab's file under cases/, rerun with --case, and check tool_log.jsonl for what changed.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.